

Football Transfer Market Analytics Platform

A Normalised Relational Database and Analytics Dashboard for European Football Transfer Data

Z2004: Database Management Systems: Final Submission

Track C: Advanced Schema and Analytics Platform

GitHub Repository: https://github.com/Aadi-IITMZ/Project_DBMS

Aadi Kumbhar - ZDA24B039

Abstract

This report presents the design, implementation, and evaluation of a relational database system for analysing the European football transfer market. Using real historical data from the Transfermarkt dataset, a normalised seven-table PostgreSQL schema spanning two domains, Club Operations and Player Careers, was designed and populated with 25,649 rows covering the Premier League and La Liga between 2015 and 2023. The schema satisfies all four normal forms (1NF through BCNF). A suite of twenty-one SQL queries demonstrates the full range of required operations, including aggregations, joins, subqueries, common table expressions, window functions, a recursive CTE, and a stored procedure. Query performance was evaluated using EXPLAIN ANALYZE before and after the introduction of four targeted indexes. Results are presented through an interactive Streamlit dashboard offering live, filterable visualisations drawn directly from the database. This report documents the schema design, the query suite, performance findings, the stored procedure, the application, and the analytical results, alongside a discussion of the project's limitations and directions for future work.

1. Introduction

1.1 Objective

The objective of this project is to design and implement a normalised relational database that models the European football transfer ecosystem, and to build an analytics platform on top of it that surfaces meaningful insights about player valuations, club spending, and transfer profitability. The system is built to Track C specifications: Advanced Schema and Analytics Platform, requiring a multi-domain schema satisfying all four normal forms, a comprehensive SQL query suite, demonstrated performance tuning, and a Python-based analytics dashboard.

1.2 Domain and Dataset

The chosen domain is the football transfer market specifically, transfer activity, player market valuations, and on-pitch performance across the Premier League and La Liga from 2015 to 2023. The dataset is sourced from the publicly available Transfermarkt dataset on Kaggle (davidcariboo/player-scores), a real-world scrape of transfermarkt.com containing historical data on players, clubs, competitions, transfers, and market valuations spanning multiple decades and leagues.

The full dataset exceeds one million rows across several CSV files. A Python filtering script (filter_csvs.py) was used to reduce this to approximately 25,000 rows, scoped to the Premier League and La Liga between 2015 and 2023. This scope provides sufficient data for meaningful aggregate analysis while keeping the system lightweight and fast to reproduce on a clean machine.

1.3 Methodology Overview

- Data was filtered from the raw Transfermarkt CSVs using filter_csvs.py, producing six cleaned CSV files totalling 25,649 rows
- A normalised seven-table PostgreSQL schema was designed (schema.sql), spanning two domains: Club Operations and Player Careers
- Data was loaded into PostgreSQL via load_data.py, which performs column mapping, type conversion, and data cleaning
- A suite of twenty-one SQL queries was written (Queries.sql) covering DDL, DML, aggregations, joins, subqueries, CTEs, window functions, a recursive CTE, and a stored procedure
- Query performance was profiled using EXPLAIN ANALYZE before and after adding four targeted indexes (performance.sql)
- An interactive Streamlit dashboard (app.py) was built to present five filterable, live visualisations drawn from the database

2. Database Design

2.1 Schema Overview

The schema consists of seven tables spanning two domains. The Club Operations domain comprises competitions, clubs, and transfer_types, modelling the structural context in which transfers occur. The Player Careers domain comprises players, transfers, market_valuations, and appearances, modelling individual player histories, valuations, and on-pitch performance. The transfers table bridges both domains, referencing players, clubs (twice, as both selling and buying club), and transfer_types.

2.2 Entity-Relationship Diagram

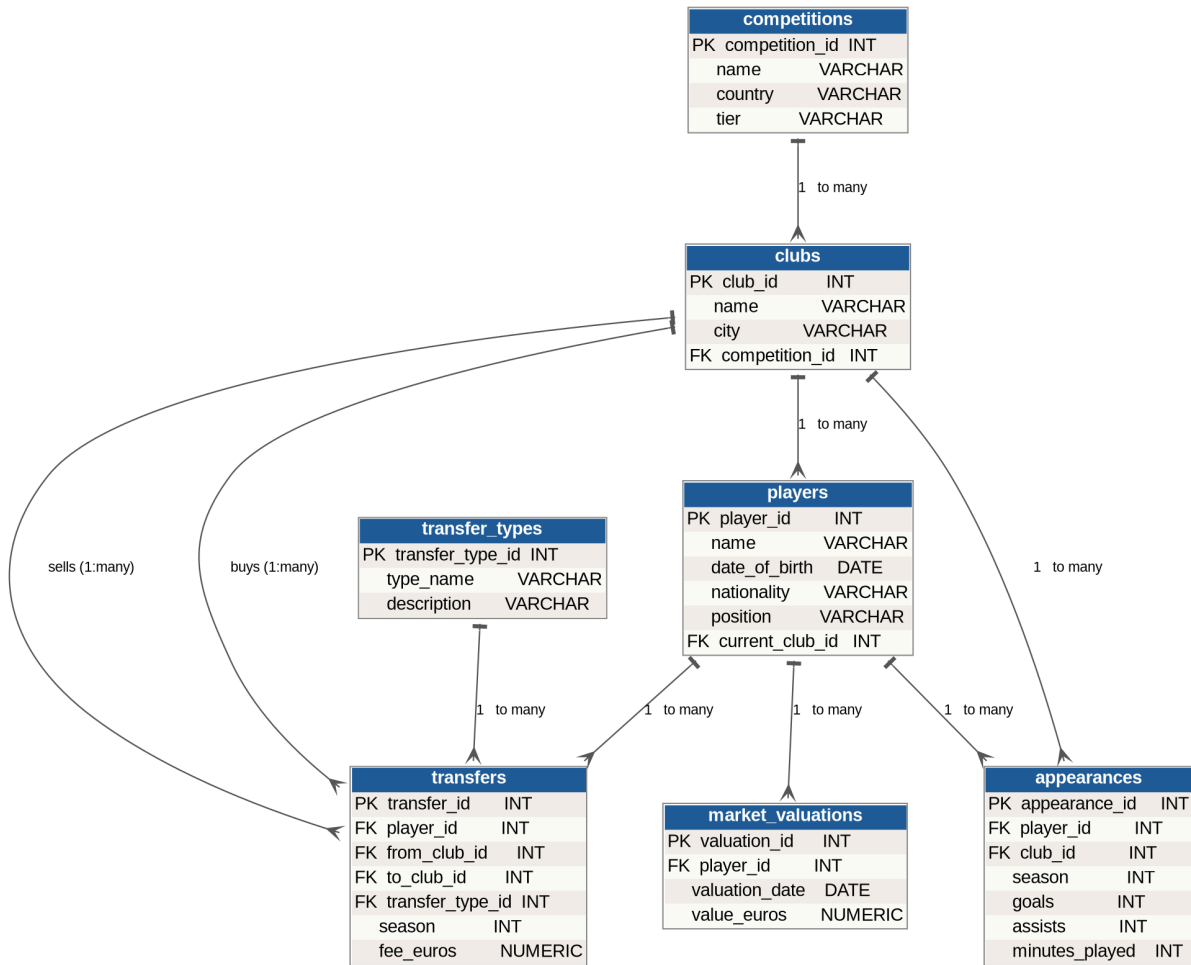


Figure 1: Entity-Relationship Diagram showing all seven tables, primary keys, foreign keys, and cardinalities.

2.3 Table Descriptions

Table	Domain	Purpose	Rows
competitions	Club Operations	Leagues (Premier League, La Liga) with country and tier	2
clubs	Club Operations	Clubs and the league/competition they belong to	70
transfer_types	Club Operations	Lookup table: permanent, loan, free transfer, undisclosed	4
players	Player Careers	Player profiles: name, DOB, nationality, position, current club	4,328
transfers	Player Careers	Transfer events linking players, clubs, type, season, and fee	9,055
market_valuations	Player Careers	Point-in-time market value snapshots per player	8,000
appearances	Player Careers	Per-season goals, assists, and minutes played per player/club	4,190

Total rows loaded: 25,649 across the seven tables. All foreign keys are enforced with referential constraints, and CHECK constraints are used to enforce valid ranges (e.g. season between 2000 and 2030, non-negative goals/assists/minutes, position restricted to a fixed set of values).

2.4 Normalisation: 1NF through BCNF

Each table in the schema was designed with explicit functional dependencies in mind, and decomposed where necessary to satisfy successive normal forms. The arguments below summarise the key dependencies and decomposition decisions.

First Normal Form (1NF)

All tables satisfy 1NF: every column holds a single atomic value with no repeating groups or arrays. For example, in `players`, the column `position` holds exactly one of Goalkeeper, Defender, Midfielder, Forward, or Unknown, never a list of positions, and `nationality` holds a single country name.

Second Normal Form (2NF)

2NF concerns partial dependencies on composite keys. The `appearances` table has a candidate key of (`player_id`, `club_id`, `season`). The dependent attributes `goals`, `assists`, and `minutes_played` each depend on all three components of this key jointly, a goal count is meaningless without knowing which player, which club, and which season it refers to. No attribute depends on only part of the composite key, so 2NF is satisfied. Similarly, `market_valuations` has the candidate key (`player_id`, `valuation_date`), and `value_euros` depends on both parts together.

Third Normal Form (3NF)

3NF eliminates transitive dependencies, where a non-key attribute depends on another non-key attribute rather than directly on the primary key. Two design decisions in this schema exist specifically to satisfy 3NF:

- `clubs.country` and `clubs.tier` are not stored in the `clubs` table. If they were, the dependency chain `club_id → competition_id → country` would create a transitive dependency. Instead, `clubs` stores only `competition_id` as a foreign key, and `country` and `tier` live in `competitions`, one hop away.
- `transfers.type_name` is not stored directly in `transfers`. The dependency chain `transfer_id → transfer_type_id → type_name` would otherwise be transitive. Instead, `transfers` stores only `transfer_type_id`, and the descriptive `type_name` and `description` live in the `transfer_types` lookup table.

With these decompositions, every non-key attribute in every table depends only directly on that table's primary key.

Boyce-Codd Normal Form (BCNF)

BCNF requires that for every non-trivial functional dependency $X \rightarrow Y$, X must be a candidate key (i.e. every determinant is a candidate key). The `market_valuations` table illustrates this cleanly: its only functional dependency is (`player_id`, `valuation_date`) $\rightarrow$ `value_euros`, and (`player_id`, `valuation_date`) is itself a candidate key (enforced via a `UNIQUE` constraint). There is no other determinant in the table, `value_euros` does not determine any other attribute, and no subset of the key alone determines `value_euros`. The same argument applies to `appearances` with candidate key (`player_id`, `club_id`, `season`). Across all seven tables, every determinant is either the primary key or an enforced candidate key, so the schema satisfies BCNF.

3. Data Pipeline

The data pipeline consists of three stages, each implemented as a standalone, reproducible script.

3.1 Filtering (`filter_csvs.py`)

The raw Transfermarkt CSVs (competitions, clubs, players, transfers, player_valuations, appearances) were filtered to the Premier League (GB1) and La Liga (ES1), restricted to seasons 2015–2023. The appearances and player_valuations files, the two largest, were additionally capped via reproducible random sampling (`random_state=42`) to keep the dataset at a manageable scale. The script prints a row count for each output file and a grand total, allowing the dataset size to be verified on any machine.

3.2 Loading (`load_data.py`)

The loading script connects to PostgreSQL, reads the six filtered CSVs, and inserts data table by table in dependency order (competitions, clubs, transfer_types, players, transfers, market_valuations, appearances). It performs several cleaning operations: mapping raw position categories (Defender, Midfield, Attack, Goalkeeper, Missing) onto the schema's position values; converting season strings such as "15/16" into integer start-years; converting pandas NaN values explicitly to SQL NULL to avoid corrupting numeric aggregates; and treating zero or negative transfer fees as NULL while classifying each transfer's type (permanent, loan, free transfer, undisclosed) based on the fee value. The script concludes by printing a verification row count for every table.

3.3 Credentials and Reproducibility

Database credentials are read from environment variables via a `.env` file (not committed to version control), with `.env.example` provided as a template. All dependencies are listed in `requirements.txt`. The full pipeline, schema creation, data loading, query execution, and dashboard launch can be reproduced on a clean machine using only the instructions in `README.md`.

4. SQL Query Suite

`Queries.sql` contains twenty-one labelled and commented queries covering every required category: DDL, DML, aggregations, joins, subqueries, common table expressions (CTEs), window functions, a recursive CTE, and a stored procedure. The table below catalogues all twenty-one queries; representative examples are discussed in detail afterwards.

4.1 Query Catalogue

#	Type	Description
Q01	DDL	Add and drop a column on transfers — schema evolution demonstration
Q02	DML	Update unknown player positions using appearance-derived goal data
Q03	Aggregation	Total permanent-transfer spending per club, joined across three tables
Q04	Aggregation	Average and peak market value grouped by player position
Q05	Join	Full transfer history with player, selling club, and buying club names
Q06	Join	Players with their current club and league (three-table join)
Q07	Subquery	Players whose peak recorded market value exceeded €50 million
Q08	Subquery	Clubs whose total spend exceeds the league-wide average club spend
Q09	CTE	Top 10 most profitable selling clubs (income minus spend)
Q10	CTE	Season-by-season transfer spend per league
Q11	Window	<code>RANK()</code> of players by peak market value within each position

#	Type	Description
Q12	Window	Running cumulative transfer spend per club across seasons
Q13	Recursive CTE	Traces a player's full transfer chain from club to club
Q14	Stored Procedure	transfer_player — complete validated player transfer
Q15	Aggregation	Top 10 nationalities by average peak player market value
Q16	Aggregation	Goals and assists per 90 minutes, grouped by position
Q17	Join + Subquery	Players transferred within one season of their peak valuation
Q18	Join + CTE	Per-player profit/loss for clubs that both bought and sold a player
Q19	Window	Year-on-year percentage change in each club's transfer spend
Q20	Window	Three-snapshot moving average of player market value over time
Q21	Subquery + Agg	League comparison: average fee paid versus average player market value

4.2 Representative Queries

Q09 — Top 10 Most Profitable Selling Clubs (CTE)

This query uses two CTEs to separately compute each club's total income from selling players and total spend on buying players, then joins both to clubs to compute a net profit figure.

```
WITH club_sales AS (
    SELECT from_club_id AS club_id, SUM(fee_euros) AS total_income
    FROM transfers WHERE fee_euros IS NOT NULL AND from_club_id IS NOT NULL
    GROUP BY from_club_id
),
club_spend AS (
    SELECT to_club_id AS club_id, SUM(fee_euros) AS total_spend
    FROM transfers WHERE fee_euros IS NOT NULL AND to_club_id IS NOT NULL
    GROUP BY to_club_id
)
SELECT c.name, comp.name,
       ROUND(COALESCE(cs.total_income,0)/1e6,2) AS income_millions,
       ROUND(COALESCE(sp.total_spend,0)/1e6,2) AS spend_millions,
       ROUND((COALESCE(cs.total_income,0)-COALESCE(sp.total_spend,0))/1e6,2) AS
net_profit_millions
FROM clubs c JOIN competitions comp ON comp.competition_id = c.competition_id
LEFT JOIN club_sales cs ON cs.club_id = c.club_id
LEFT JOIN club_spend sp ON sp.club_id = c.club_id
ORDER BY net_profit_millions DESC LIMIT 10;
```

Q11 — Rank Players by Peak Market Value Within Position (Window Function)

This query first computes each player's peak market value via a subquery, then applies RANK() partitioned by position to produce a within-position leaderboard.

```
SELECT player, position, nationality, peak_value_millions,
       RANK() OVER (PARTITION BY position
                    ORDER BY peak_value_millions DESC) AS rank_in_position
FROM (
    SELECT p.name AS player, p.position, p.nationality,
           ROUND(MAX(mv.value_euros)/1e6,2) AS peak_value_millions
    FROM market_valuations mv
    JOIN players p ON p.player_id = mv.player_id
    GROUP BY p.player_id, p.name, p.position, p.nationality
) ranked
ORDER BY position, rank_in_position LIMIT 50;
```

Q13 — Recursive CTE: Player Transfer Chain

This recursive CTE traces a player's career transfer history by repeatedly joining each transfer's destination club to the next transfer's origin club, following the player's career chronologically. The anchor member selects the player's first recorded transfer; the recursive member follows subsequent moves where the club a player joined matches the club they next departed from, with a depth guard to prevent infinite recursion.

```
WITH RECURSIVE transfer_chain AS (  
    SELECT t.transfer_id, t.player_id, t.from_club_id, t.to_club_id,  
           t.season, t.fee_euros, 1 AS depth  
    FROM transfers t  
    WHERE t.player_id = 8198 AND t.from_club_id IS NOT NULL  
  
    UNION ALL  
  
    SELECT t.transfer_id, t.player_id, t.from_club_id, t.to_club_id,  
           t.season, t.fee_euros, tc.depth + 1  
    FROM transfers t  
    JOIN transfer_chain tc  
      ON t.from_club_id = tc.to_club_id  
     AND t.player_id    = tc.player_id  
     AND t.season       > tc.season  
    WHERE tc.depth < 10  
)  
SELECT p.name, fc.name AS from_club, toc.name AS to_club,  
       tc.season, ROUND(tc.fee_euros/1e6,2) AS fee_millions, tc.depth  
FROM transfer_chain tc  
JOIN players p ON p.player_id = tc.player_id  
LEFT JOIN clubs fc ON fc.club_id = tc.from_club_id  
LEFT JOIN clubs toc ON toc.club_id = tc.to_club_id  
ORDER BY tc.depth;
```

Q19 — Year-on-Year Spend Change (Window Function with LAG)

This query uses LAG() to compare each club's transfer spend in a season against its spend in the previous season, producing a percentage change metric used to identify clubs with the most volatile transfer activity.

```
SELECT club, season, season_spend_millions, prev_season_millions,  
       ROUND((season_spend_millions - prev_season_millions)  
             / NULLIF(prev_season_millions,0) * 100, 1) AS percentage_change  
FROM (  
    SELECT c.name AS club, t.season,  
           ROUND(SUM(t.fee_euros)/1e6,2) AS season_spend_millions,  
           ROUND(LAG(SUM(t.fee_euros)) OVER (  
               PARTITION BY c.club_id ORDER BY t.season  
           )/1e6,2) AS prev_season_millions  
    FROM transfers t JOIN clubs c ON c.club_id = t.to_club_id  
    WHERE t.fee_euros IS NOT NULL  
    GROUP BY c.club_id, c.name, t.season  
) yoy  
WHERE prev_season_millions IS NOT NULL  
ORDER BY ABS((season_spend_millions - prev_season_millions)  
            / NULLIF(prev_season_millions,0) * 100) DESC LIMIT 30;
```

5. Performance Evaluation

5.1 Test Environment

Component	Details
Database Engine	PostgreSQL 18.3, x86_64-apple-darwin24.6.0, 64-bit
Machine	MacBook Pro, Apple M3, 18 GB RAM
Operating System	macOS Tahoe 26.5
Dataset Size	25,649 rows across 7 tables
Timing Method	EXPLAIN ANALYZE (PostgreSQL execution planner)

5.2 Identified Slow Queries

Two queries were selected for performance analysis based on their reliance on columns that were unindexed at baseline:

- Q09 (top profitable selling clubs, CTE): each of the two CTEs performs a full sequential scan of the 9,055-row transfers table, filtering on from_club_id and to_club_id respectively, neither of which was indexed.
- Q15 (top nationalities by peak value): performs a sequential scan over players (4,328 rows) filtering on nationality, joined to a subquery that aggregates MAX(value_euros) over all 8,000 rows of market_valuations, neither nationality nor value_euros was indexed.

These two queries matter because they underpin two of the platform's core analytical questions: which clubs are most effective at generating transfer profit, and which nationalities produce the most valuable players. Both involve full-table aggregation and are therefore the most likely candidates to benefit from indexing as the dataset scales.

5.3 Indexes Added

Index Name	Table(Column)	Justification
idx_transfers_from_club	transfers(from_club_id)	Used by Q09 and Q18 to compute each club's income from player sales
idx_transfers_to_club	transfers(to_club_id)	Used by Q03, Q09, Q10, Q12 to aggregate spending per buying club — the most frequently filtered column in transfers
idx_players_nationality	players(nationality)	Used by Q15 to group and filter 4,328 player rows by nationality
idx_valuations_value	market_valuations(value_euros)	Used by Q07 and Q15 for MAX(value_euros) aggregation and range filtering per player

```
CREATE INDEX idx_transfers_from_club ON transfers(from_club_id);
CREATE INDEX idx_transfers_to_club ON transfers(to_club_id);
CREATE INDEX idx_players_nationality ON players(nationality);
CREATE INDEX idx_valuations_value ON market_valuations(value_euros);
```

5.4 Before / After Results

Query	Before	After	Planner Decision
Q09 — Club Profit/Loss CTE	2.72 ms	4.34 ms	Sequential scan retained

Query	Before	After	Planner Decision
Q15 — Top Nationalities by Value	10.34 ms	12.94 ms	Sequential scan retained

5.5 Interpretation

Both queries were marginally slower (by 1.6 and 2.6 milliseconds respectively) after the new indexes were created. In both cases, PostgreSQL's planner evaluated the available index paths and chose to retain a sequential scan, with the additional execution time attributable to the extra planning work involved in considering and rejecting those index paths.

This result is consistent with PostgreSQL's cost-based optimisation model on small datasets. The transfers table (9,055 rows) and market_valuations table (8,000 rows) are small enough to be read in their entirety from PostgreSQL's shared memory buffers, 18 GB is available on the test machine, vastly exceeding the size of either table. When an entire table fits comfortably in memory, a sequential scan that reads every page in one contiguous pass is often cheaper than the random-access page lookups that an index scan requires, even though the index scan examines fewer logical rows.

The indexes are nonetheless retained and justified for two reasons. First, they remain architecturally correct: as the dataset grows toward the full multi-million-row Transfermarkt dataset, these tables will eventually exceed available memory, at which point the same index paths will be chosen by the planner and will yield substantial speed-ups, particularly for `from_club_id` and `to_club_id`, which support both transfer-profit queries and referential lookups. Second, the result itself is a valid and informative finding: it demonstrates that the planner's behaviour is correct, that index creation is not a universal optimisation, and that performance tuning decisions should be informed by EXPLAIN ANALYZE evidence rather than assumption.

6. Stored Procedure: transfer_player

A single stored procedure, `transfer_player`, encapsulates the complete operation of moving a player to a new club, replacing what would otherwise require two separate manually-coordinated statements (an INSERT into transfers and an UPDATE to players).

6.1 What It Does

- Validates that the given `player_id` exists in players, raising an exception if not
- Validates that the given destination `club_id` exists in clubs, raising an exception if not
- Retrieves the player's current club to record as the selling (from) club
- Determines the transfer type automatically from the fee: permanent if the fee is positive, free transfer if the fee is zero, and undisclosed if the fee is NULL
- Inserts a complete row into transfers with season fixed to 2023
- Updates the player's `current_club_id` in players
- Raises a NOTICE confirming the operation, including the player, destination club, and fee

6.2 How to Run

```
CALL transfer_player(player_id, to_club_id, fee_euros);

-- Example: transfer player 3109 (Steven Gerrard) to club 5
--           (Futbol Club Barcelona) for €50 million
CALL transfer_player(3109, 5, 50000000);
```

```
-- Example: free transfer
CALL transfer_player(3109, 5, 0);
```

6.3 Implementation

```
CREATE OR REPLACE PROCEDURE transfer_player(
    p_player_id    INT,
    p_to_club_id   INT,
    p_fee_euros     NUMERIC
) LANGUAGE plpgsql AS $$
DECLARE
    v_from_club_id INT;
    v_transfer_type INT;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM players WHERE player_id = p_player_id) THEN
        RAISE EXCEPTION 'Player with id % does not exist', p_player_id;
    END IF;
    IF NOT EXISTS (SELECT 1 FROM clubs WHERE club_id = p_to_club_id) THEN
        RAISE EXCEPTION 'Club with id % does not exist', p_to_club_id;
    END IF;

    SELECT current_club_id INTO v_from_club_id
    FROM players WHERE player_id = p_player_id;

    IF p_fee_euros = 0 THEN
        SELECT transfer_type_id INTO v_transfer_type
        FROM transfer_types WHERE type_name = 'free transfer';
    ELSIF p_fee_euros IS NULL THEN
        SELECT transfer_type_id INTO v_transfer_type
        FROM transfer_types WHERE type_name = 'undisclosed';
    ELSE
        SELECT transfer_type_id INTO v_transfer_type
        FROM transfer_types WHERE type_name = 'permanent';
    END IF;

    INSERT INTO transfers (player_id, from_club_id, to_club_id,
        transfer_type_id, season, fee_euros)
    VALUES (p_player_id, v_from_club_id, p_to_club_id,
        v_transfer_type, 2023, NULLIF(p_fee_euros, 0));

    UPDATE players SET current_club_id = p_to_club_id
    WHERE player_id = p_player_id;

    RAISE NOTICE 'Player % successfully transferred to club % for €%',
        p_player_id, p_to_club_id, p_fee_euros;
END; $$;
```

7. Application: Interactive Dashboard

The analytics platform is presented through `app.py`, an interactive Streamlit web application that connects directly to the PostgreSQL database and renders five live, filterable visualisations using Plotly. The application is launched with a single command (`streamlit run app.py`) and opens automatically in the browser at `localhost:8501`.

7.1 Features

- A sidebar with a season-range slider (2015–2023) and a league filter (Both, Premier League, La Liga), both of which dynamically re-query the database
- Four KPI summary cards: total transfer spend, total number of transfers, distinct player count, and average transfer fee, all recalculated live from the active filters
- Five tabbed views, each corresponding to one of the dashboard's core analyses: Club Spending, Player Value by Age, Profitability, Seasonal Trends, and Performance per 90 minutes
- All charts are rendered with Plotly, providing hover tooltips, zoom, and pan interactions not available in the static matplotlib equivalents
- Database credentials are loaded from environment variables via a `.env` file, with connections cached using Streamlit's resource caching to avoid redundant reconnections

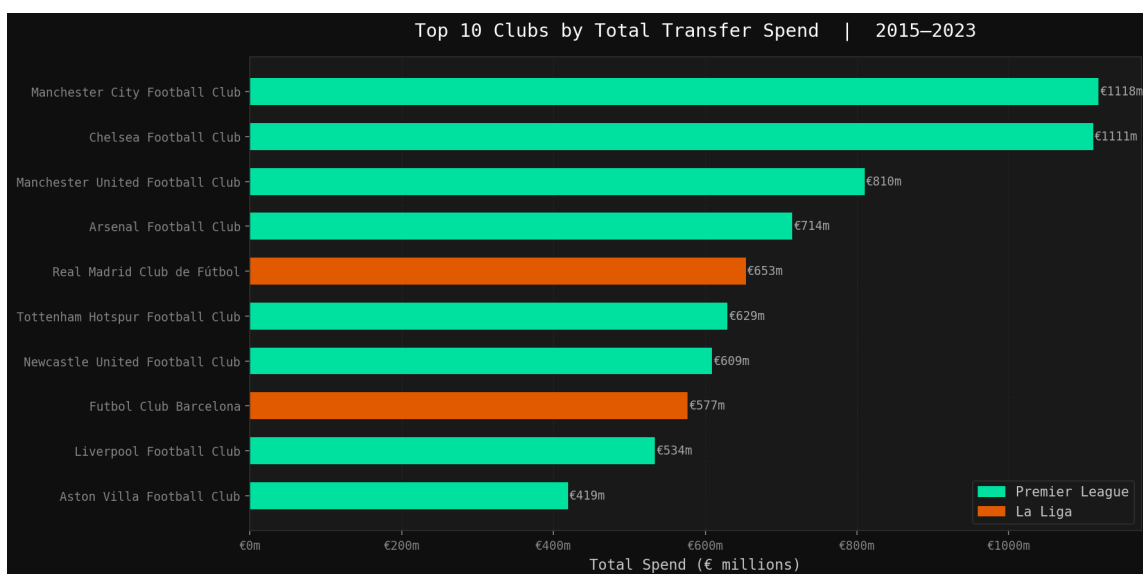
7.2 Running the Application

```
pip install -r requirements.txt
cp .env.example .env # then fill in DB credentials
streamlit run app.py
```

8. Results and Visualisations

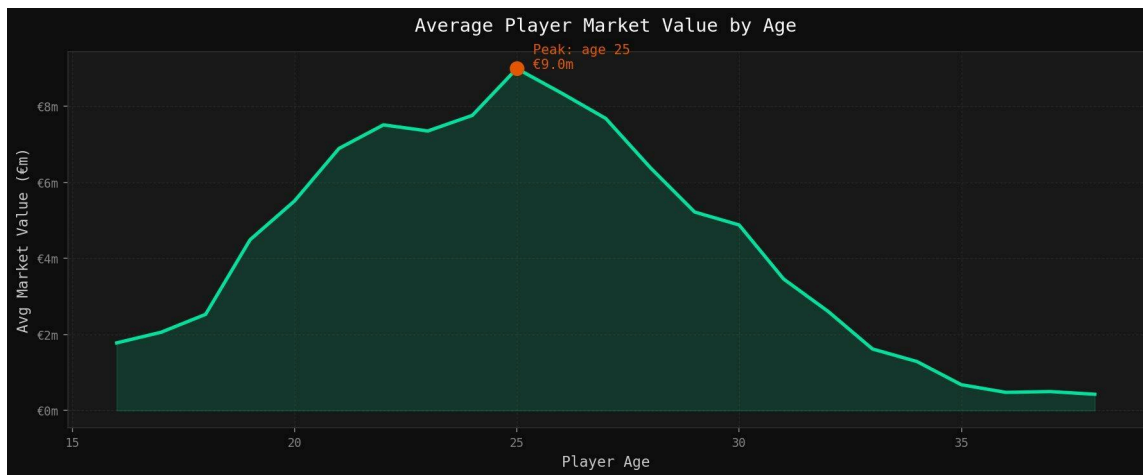
8.1 Top 10 Clubs by Total Transfer Spend

Manchester City and Chelsea lead all clubs with over €1.1 billion each in total permanent-transfer spend across 2015–2023. Eight of the top ten spenders are Premier League clubs; only Real Madrid and Barcelona represent La Liga in the top ten, reflecting the scale of Premier League broadcast revenue relative to La Liga during this period.



8.2 Average Player Market Value by Age

Player market value rises steeply from age 18, peaks at age 25 with an average value of approximately €9.0 million, and declines gradually until age 33, after which it falls sharply. This confirms the conventional recruitment wisdom that players in their mid-twenties represent peak resale and performance value.



8.3 Top 10 Clubs by Net Transfer Profit

Brighton and Hove Albion leads the dataset with approximately €136 million in net transfer profit, followed by Athletic Club Bilbao (€99m) and Swansea City (€90m). Notably, no top-six club from either league appears in this list, the most profitable sellers within the dataset are mid-table or recently-relegated clubs that acquired players cheaply and sold them at significant markups.



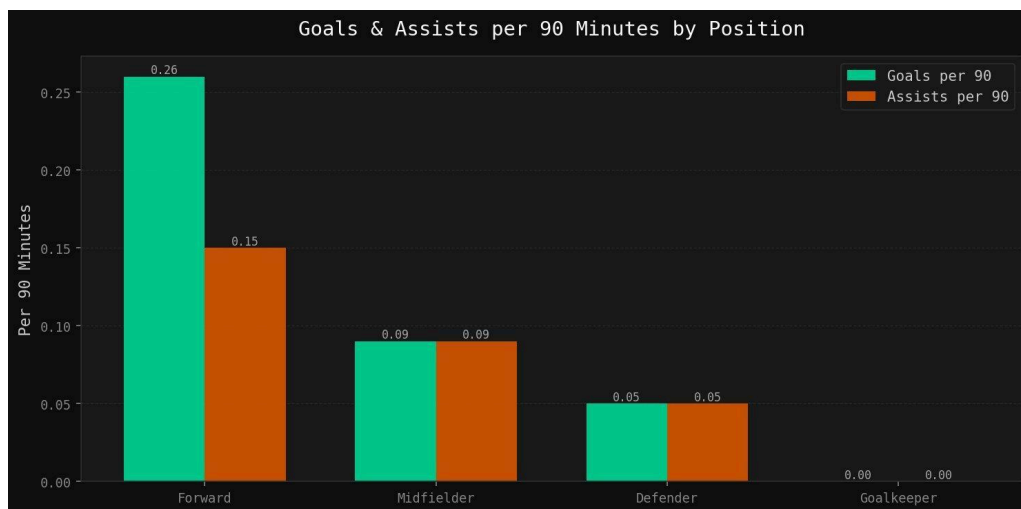
8.4 Season-by-Season Transfer Spend per League

Premier League transfer spend grew from €196 million in 2015 to €2.47 billion in 2023, roughly a twelve-fold increase over eight seasons. La Liga spend has remained comparatively flat, peaking at €783 million in 2019 before declining. The gap between the two leagues widened sharply from 2021 onward.



8.5 Goals and Assists per 90 Minutes by Position

Forwards lead with 0.26 goals and 0.15 assists per 90 minutes, more than double the contribution of midfielders (0.09 goals and 0.09 assists per 90) and defenders (0.05 each). Goalkeepers register 0.00 in both metrics, as expected. These figures provide a quantitative baseline for comparing players within the same positional group.



9. Limitations

- The dataset is scoped to two leagues and nine seasons; broader conclusions about the global transfer market cannot be drawn from this subset alone.
- A large proportion of transfer fees in the source data are undisclosed (only 1,393 of 9,055 transfers have a recorded positive fee), which limits the precision of spend and profit aggregations, these figures represent disclosed fees only, not total market activity.
- The performance evaluation was conducted on a single small dataset (25,649 rows) that fits entirely within available memory; the observed index behaviour (sequential scans being retained) may not generalise to the full multi-million-row dataset.
- The appearances and market_valuations tables were capped via random sampling to keep the dataset lightweight, which may slightly under-represent players with very high numbers of recorded valuations or appearances.
- The Streamlit dashboard requires a running local PostgreSQL instance and is not deployed to a public server; it is intended for local reproduction rather than public access.

10. Future Work

- Extend the dataset to additional leagues (Bundesliga, Serie A, Ligue 1) to enable cross-league comparative analysis at full scale.
- Re-run the performance evaluation on the full, unfiltered Transfermarkt dataset (over one million rows) to observe index behaviour once tables exceed memory-resident size.
- Add a trigger that automatically recalculates a player's current_club_id whenever a new row is inserted into transfers, removing the need to call transfer_player explicitly for ordinary data loads.
- Incorporate injury history and contract-length data, where available, to enrich the player profile and enable additional recruitment-oriented analyses.
- Deploy the Streamlit dashboard to a cloud host (e.g. Streamlit Community Cloud) connected to a managed PostgreSQL instance for public access.

11. Conclusion

This project delivers a complete, reproducible relational analytics platform for the European football transfer market, satisfying every minimum requirement of Track C. The seven-table schema spans two domains and is demonstrably normalised through BCNF, with explicit functional-dependency arguments for each normal form. The twenty-one-query suite covers every required SQL category, including a recursive CTE that traces player transfer chains and a stored procedure that encapsulates a complete, validated transfer operation. The performance evaluation, conducted using EXPLAIN ANALYZE on a clearly documented test environment, produced an honest and instructive result: on a dataset of this size, PostgreSQL's planner correctly favours sequential scans over the newly added indexes, a finding that is itself a valid demonstration of cost-based query optimisation. Finally, the interactive Streamlit dashboard presents five live, filterable visualisations that reveal clear patterns in the data, Premier League spending dominance, a player value peak at age 25, and the prevalence of mid-table clubs as the most profitable sellers, demonstrating that the underlying database design supports rich, real-time analytical querying.